



Vietnam National University of HCMC
International University
School of Computer Science and Engineering



Object Oriented Programming

Building Objects' Relationships

Instructor: Le Thi Ngoc Hanh, Ph.D

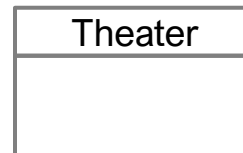
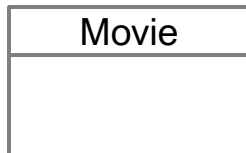
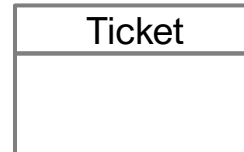
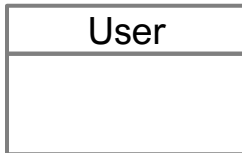
ltnhanh@hcmiu.edu.vn

Semester – Spring 2026

Case study

A cinema company is developing a simple system to manage movie ticket purchases. Each User represents a customer who can purchase tickets. Each Ticket represents a purchased seat for a specific movie. A ticket must be associated with exactly one user and cannot exist without a user. Each Movie represents a film in the system. A movie exists independently and can have multiple tickets associated with it. Each theater contains multiple seats. Seats do not exist outside a theater.

Suggested Classes



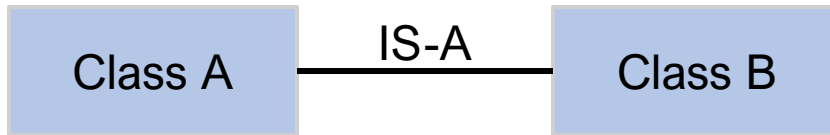
Lecture Objectives

Relationship and Multiplicity

Introduction

- ✧ In real-world systems, entities do not exist independently. Instead, they interact with each other to achieve functionality.
- ✧ A system is best understood as a graph of objects, where nodes are objects and edges are relationships
- ✧ Object relationships allow us to:
 - ▶ Divide complex systems into manageable components
 - ▶ Represent real-world dependencies accurately
 - ▶ Enable reuse and separation of concerns

Introduction



What if without IS-A?

Has-a Relationship

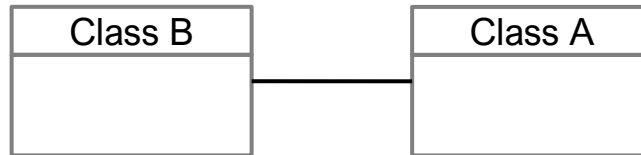
- ☾ The has-a relationship is an implementation concept where one class contains another class as a field. This is typically implemented using object references.
- ☾ The has-a relationship itself does not specify what kind of has-a relationship; that depends on how the object is created and managed.

Association

- ☾ Association is the most general form of relationship between two classes.
- ☾ Association is a fundamental structural relationship in object-oriented modeling that describes how objects of different classes are connected and interact with each other.
- ☾ It simply indicates that one object is aware of or interacts with another. There is **no ownership** implied, and both objects can exist independently of each other.
- ☾ This type of relationship is commonly used when objects need to communicate but do not control each other's lifecycle.

Association

★ UML representation:



★ Sample code:

```
class B {
    String name;
    B(String name) {
        this.name = name;
    }
}
```

```
class A {
    B b;
    void show() {
        System.out.println("A uses B: " + b.name);
    }
}
```

Association

A blue arrow points from the text 'Association' to the line 'B b;' in the sample code for class A.

Association – Sample

```
class Main {  
    public static void main(String[] args) {  
        // Step 1: Create B independently  
        B b = new B("Object B1");  
        // Step 2: Create A  
        A a = new A();  
        // Step 3: Link A to B (association)  
        a.b = b;  
        // Step 4: Use A  
        a.show();  
        // Step 5: Remove A  
        a = null;  
        // B still exists independently  
        System.out.println("B still exists: " + b.name);  
    }  
}
```

```
class Seat {  
    private String seatNumber;  
    public Seat(String seatNumber) {  
        this.seatNumber = seatNumber;  
    }  
    public String getSeatNumber() {  
        return seatNumber;  
    }  
}
```

```
class Ticket {  
    private String ticketId;  
    private Seat seat; // Association  
    public Ticket(String ticketId, Seat seat) {  
        this.ticketId = ticketId;  
        this.seat = seat; // reference only  
    }  
    public void printTicket() {  
        System.out.println("Ticket ID: " + ticketId);  
        System.out.println("Seat: " + seat.getSeatNumber());  
    }  
}
```

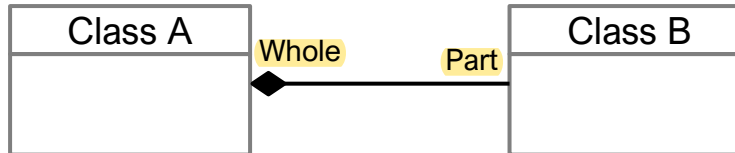


Association – Sample

```
public class Main {  
    public static void main(String[] args) {  
        // Seat is created independently  
        Seat seat = new Seat("A10");  
  
        // Ticket only references Seat  
        Ticket ticket = new Ticket("T001", seat);  
  
        ticket.printTicket();  
    }  
}
```

Composition

- Composition is strong Whole-Part relationship, where the part **cannot exist independently** of the whole.
- In this relationship, the whole object is responsible for creating and destroying its parts. This implies a lifecycle dependency, meaning if the whole is destroyed, all its parts are also destroyed.
- UML representation



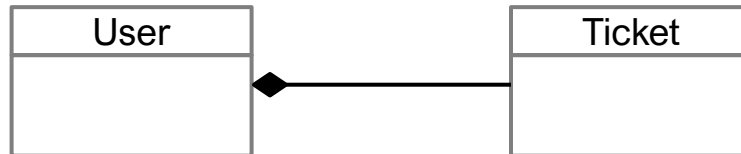
```
class B {  
    private String name;  
    public B(String name) {  
        this.name = name;  
    }  
    public String getName() {  
        return name;  
    }  
}  
  
class A {  
    private List<B> listB = new ArrayList<>();  
  
    // A creates B internally  
    public void createB(String name) {  
        B b = new B(name); // created inside A  
        listB.add(b);  
    }  
  
    public void showB() {  
        for (B b : listB) {  
            System.out.println("B: " + b.getName());  
        }  
    }  
}
```



```
class Main {  
    public static void main(String[] args) {  
  
        // Step 1: Create A  
        A a = new A();  
  
        // Step 2: A creates B objects internally  
        a.createB("Object B1");  
        a.createB("Object B2");  
  
        // Step 3: Use A  
        a.showB();  
  
        // Step 4: Remove A  
        a = null;  
  
        // B objects are no longer accessible --> lifecycle ended with A  
        System.out.println("A deleted --> B objects are also gone");  
    }  
}
```

Sample: User & Ticket

```
class Ticket {  
    private String ticketId;  
    public Ticket(String ticketId) {  
        this.ticketId = ticketId;  
    }  
    public String getTicketId() {  
        return ticketId;  
    }  
}
```



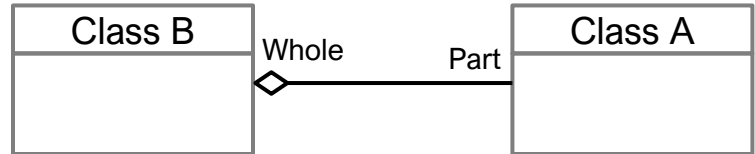
```
class User {  
    private String userId;  
    private List<Ticket> tickets = new ArrayList<>();  
  
    public User(String userId) {  
        this.userId = userId;  
    }  
    // User creates Ticket internally  
    public void purchaseTicket(String ticketId) {  
        Ticket ticket = new Ticket(ticketId); // created inside  
        tickets.add(ticket);  
    }  
    public void showTickets() {  
        System.out.println("User: " + userId);  
        for (Ticket t : tickets) {  
            System.out.println("Ticket ID: " + t.getTicketId());  
        }  
    }  
}
```

```
class Main {  
    public static void main(String[] args) {  
  
        // Create User  
        User user = new User("U001");  
  
        // User creates and owns Tickets  
        user.purchaseTicket("T001");  
        user.purchaseTicket("T002");  
  
        user.showTickets();  
  
        // Remove User  
        user = null;  
  
        // Tickets no longer accessible --> lifecycle ended with User  
        System.out.println("User deleted --> Tickets are also gone");  
    }  
}
```

Aggregation (Weak Whole-Part Relationship)

- ☾ Aggregation represents a whole-part relationship, where one object contains other objects, but the contained objects can still exist independently. This means the lifecycle of the part is not controlled by the whole.
- ☾ Aggregation is often used when objects are shared across multiple systems or contexts.

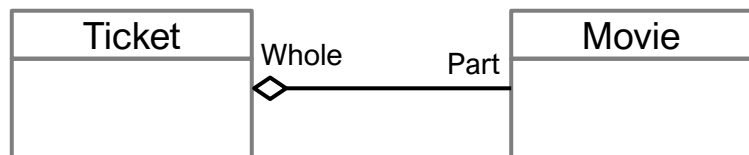
```
class B {  
    private String name;  
    public B(String name) {  
        this.name = name;  
    }  
    public String getName() {  
        return name;  
    }  
}
```



```
class A {  
    private List<B> listB; // Aggregation  
  
    // B objects are passed from outside  
    public A(List<B> listB) {  
        this.listB = listB;  
    }  
  
    public void showB() {  
        for (B b : listB) {  
            System.out.println("B: " + b.getName());  
        }  
    }  
}
```

```
class Main {  
    public static void main(String[] args) {  
  
        // Step 1: Create B objects independently  
        B b1 = new B("Object B1");  
        B b2 = new B("Object B2");  
  
        // Step 2: Inject B objects into A  
        A a = new A(Arrays.asList(b1, b2));  
  
        // Step 3: Use A  
        a.showB();  
  
        // Step 4: Remove A  
        a = null;  
  
        // B objects still exist  
        System.out.println("B1 still exists: " + b1.getName());  
    }  
}
```

```
class Ticket {  
    private String ticketId;  
  
    public Ticket(String ticketId) {  
        this.ticketId = ticketId;  
    }  
  
    public String getTicketId() {  
        return ticketId;  
    }  
}
```



```
class Movie {  
    private String title;  
    private List<Ticket> tickets;  
  
    public Movie(String title, List<Ticket> tickets) {  
        this.title = title;  
        this.tickets = tickets; // injected from outside  
    }  
  
    public void showTickets() {  
        System.out.println("Movie: " + title);  
        for (Ticket t : tickets) {  
            System.out.println("Ticket ID: " + t.getTicketId());  
        }  
    }  
}
```

Comparison of Relationships

- ✧ Understanding the difference between association, aggregation, and composition is critical for proper system design.
- ✧ These relationships differ mainly in terms of ownership and lifecycle dependency.

Relationship	Ownership	Lifecycle
Association	None	Independent
Aggregation	Weak	Independent
Composition	Strong	Dependent

Objects' relationship

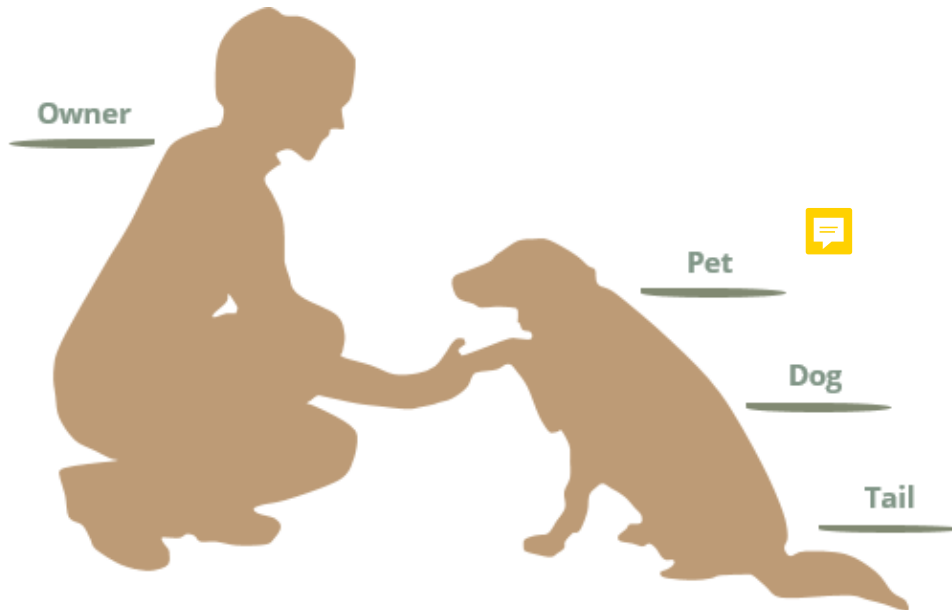
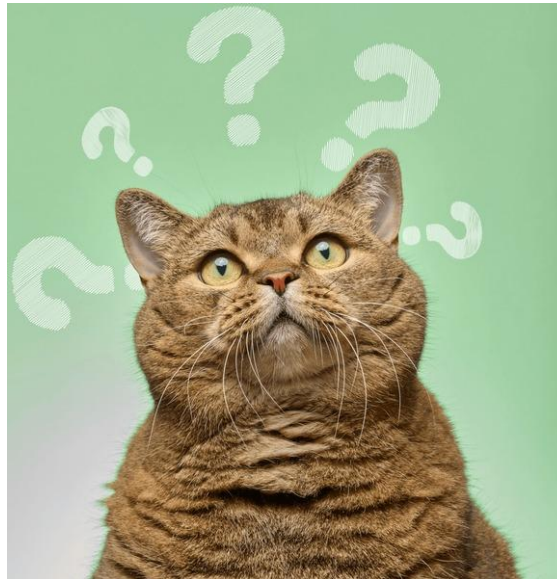


Image source: <https://www.visual-paradigm.com>

WHAT kind of relationship



HOW MANY?



Multiplicity

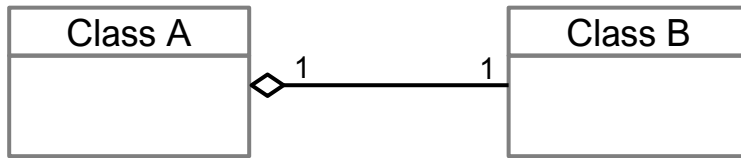
- ✧ Multiplicity (also called *cardinality*) defines how many objects of one class can be associated with a single object of another class in an object relationship.
- ✧ Multiplicity is a fundamental concept in UML class diagrams because it specifies the quantitative constraints of a relationship, not just its existence.
- ✧ Multiplicity provides this precision and helps avoid ambiguity in both design and implementation.

One-to-One multiplicity

- ★ A one-to-one (1-1) multiplicity indicates that each object of one class is associated with exactly one object of another class, and vice versa. This means the relationship is exclusive on both sides: no object can be linked to more than one counterpart.
- ★ In a one-to-one composition, a single object contains exactly one instance of another object. This is used when the part is essential to the whole and cannot exist independently.

One-to-One multiplicity

UML representation



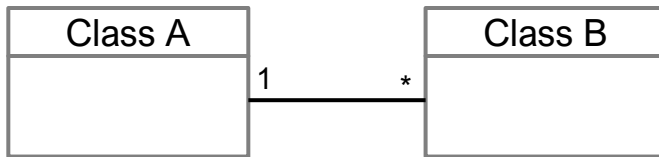
```
class B {...}
```

```
class A {  
    B b;  
}
```

One-to-Many multiplicity (1-N)

- ★ A one-to-many multiplicity occurs when a single object is associated with multiple objects of another class. This is commonly implemented using collections such as `List`, `Set`, or `Map`. In this relationship, a single object owns multiple dependent objects. These objects are typically stored in a collection and are created and managed by the owning class.

- ★ UML representation

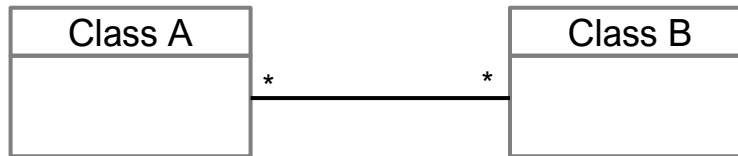


```
class B {...}
```

```
class A {
    List<B> list = new ArrayList<>();
}
```

Many-to-Many multiplicity (M-N)

- ★ In a many-to-many relationship, multiple objects of one class are associated with multiple objects of another class. This relationship is more complex and often requires careful management to maintain consistency.



```
class Student {  
    private List<Course> courses = new ArrayList<>();  
}  
  
class Course {  
    private List<Student> students = new ArrayList<>();  
}
```

Many-to-Many multiplicity (M-N)

- ★ In real systems, many-to-many is often transformed into two one-to-many relationships using an intermediate class:

